100+ C# interview questions with concise answers (Basic → Medium)

🔹 1. Classes and Objects (10 Q&A)

1. What is a class?
A class is a blueprint for creating objects. It defines properties and methods.

2. What is an object?
An object is an instance of a class containing data and behavior.

3. How do you create an object?

```
Student s = new Student();
```

4. Difference between class and object?
Class = blueprint, Object = instance of that blueprint.

5. What is encapsulation?
Wrapping data and methods into a single unit and restricting access using access modifiers.

6. What is a static class?
A class that cannot be instantiated and contains only static members.

7. Can a class have multiple objects?
Yes, unlimited objects.

8. What are instance members?
Members that belong to object instances.

9. What are static members?
Shared members across all objects.

10. Can a class exist without objects?
Yes, but it's useful only when objects are created.

🔹 2. Constructors and Properties (10 Q&A)

11. What is a constructor?
A special method called when an object is created.

12. Types of constructors?
Default, parameterized, copy, static.

13. What is a default constructor?
Constructor with no parameters.

14. Can constructors be overloaded?
Yes.

15. What is constructor chaining?
Calling one constructor from another using this().

16. What is a property?
A member that provides controlled access to fields.

17. What are auto-properties?

public int Age { get; set; }

18. Read-only property?
Property with only get.

19. Can constructor be private?
Yes, used in Singleton pattern.

20. Difference between property and field?
Field = variable, Property = controlled access.

🔹 3. Access Modifiers (10 Q&A)

21. What are access modifiers?
Keywords that define accessibility.

22. Types?
public, private, protected, internal, protected internal, private protected

23. Default for class?
internal

24. Default for members?
private

25. public vs private?
public = accessible everywhere, private = within class only

26. protected?
Accessible in derived classes.

27. internal?
Accessible within same assembly.

28. protected internal?
Accessible in same assembly OR derived classes.

29. private protected?
Derived classes within same assembly.

30. Why use access modifiers?
For data hiding and security.

- **4. Class Variables / Fields (10 Q&A)**

31. What is a field?
A variable declared in a class.

32. Default values?
int = 0, bool = false, reference = null

33. static field?
Shared across all objects.

34. instance field?
Unique per object.

35. const vs readonly?
const = compile-time, readonly = runtime

36. Where is const stored?
In metadata.

37. Can readonly change?
Yes, only in constructor.

38. What is field initialization?
Assigning value at declaration.

39. Difference local vs field?
Local = inside method, field = class level.

40. Can fields be private?
Yes (recommended).

🔹 5. Inheritance (10 Q&A)

41. What is inheritance?
Acquiring properties from another class.

42. Syntax?

class Child : Parent {}

43. Types in C#?
Single, Multilevel, Hierarchical

44. Does C# support multiple inheritance?
No (via classes), yes via interfaces.

45. What is base keyword?
Access base class members.

46. What is method overriding?
Changing base method behavior.

47. virtual keyword?
Allows overriding.

48. sealed class?
Cannot be inherited.

49. Why use inheritance?
Code reuse.

50. What is IS-A relationship?
Represents inheritance.

🔹 6. Polymorphism (10 Q&A)

51. What is polymorphism?
One interface, multiple implementations.

52. Types?
Compile-time & Runtime.

53. Method overloading?
Same method name, different parameters.

54. Method overriding?
Redefining base method.

55. virtual vs override?
virtual = base, override = derived.

56. new keyword?
Method hiding.

57. Operator overloading?
Custom behavior for operators.

58. Can overloading differ by return type?
No.

59. Early binding?
Compile-time.

60. Late binding?
Runtime.

- **7. Exception Handling (10 Q&A)**

61. What is exception?
Runtime error.

62. try-catch?
Handles exceptions.

63. finally block?
Always executes.

64. throw keyword?
Used to raise exception.

65. Exception class?
Base class for all exceptions.

66. Custom exception?
User-defined exception.

67. Difference throw vs throw ex?
throw preserves stack trace.

68. Multiple catch blocks?
Yes.

69. When use finally?
Cleanup code.

70. Can we skip catch?
Yes, try-finally.

🔹 8. Collections (10 Q&A)

71. What are collections?
Classes to store multiple objects.

72. Types?
List, Dictionary, Stack, Queue, HashSet

73. List?
Dynamic array.

74. Dictionary?
Key-value pairs.

75. Stack?
LIFO.

76. Queue?
FIFO.

77. HashSet?
Unique elements.

78. Difference Array vs List?
Array fixed, List dynamic.

79. Generic collections?
Type-safe.

80. Why use collections?
Efficient data handling.

🔹 9. File I/O (10 Q&A)

81. What is File I/O?
Reading/writing files.

82. Classes used?
File, FileStream, StreamReader, StreamWriter

83. Write file?

File.WriteAllText("a.txt","Hello");

84. Read file?

File.ReadAllText("a.txt");

85. StreamReader?
Reads text.

86. StreamWriter?
Writes text.

87. FileMode?
Controls file opening.

88. What is buffering?
Temporary memory storage.

89. Difference File vs FileStream?
File = helper, FileStream = low-level.

90. Exception in file handling?
IOException.

- **10. Nullable Types (5 Q&A)**

91. What is nullable type?
Allows value types to be null.

92. Syntax?

int? x = null;

93. Why needed?
Handle missing data.

94. HasValue?
Checks value exists.

95. Value property?
Gets actual value.

🔹 11. Structures and Enums (10 Q&A)

96. What is struct?
Value type for small data.

97. Difference class vs struct?
Class = reference, struct = value.

98. Can struct have constructor?
Yes (no parameterless explicitly).

99. What is enum?
Named constants.

100. Example enum?

```
enum Days { Mon, Tue }
```

101. Default value of enum?
0

102. Can enum have methods?
No

103. When to use struct?
Small, lightweight objects.

104. Boxing in struct?
Converting value to object.

105. Unboxing?
Object back to value.